算法一：

按要求使用暴力或者 KMP 模拟。

时间复杂度：$O(N^2)$或 $O(N^3)$。

期望得分：10-20 分。

算法二：

另有 10%的数据满足：字符串 S 仅由字母“n”组成。

经过一些数学推导可得出 ans=$\sum_{k=1}^{n}\frac{k(k+1)}{2}*(n-k+1)$，预处理 $\sum_{k=1}^{n}\frac{k(k+1)}{2}*(k\text{-}1)$ 与 $\sum_{k=1}^{n}\frac{k(k+1)}{2}$ 的结果即可。

时间复杂度：O(N)。

期望得分：20-30 分（结合算法一）。

算法三：

另有 30%的数据满足：字符串 S 为随机生成。

我们重新审视一下 KMP 树上的深度的性质，不难发现 KMP 树上节点|S|的深度即为有多少 S 的前缀等于 S 的后缀。

如果我们已经得到了 1<=l<=r<=M 的 f(l:r)的答案，考虑如何得到 1<=l<=r<=M+1 的答案。维护子串[1,M]、[2,M]、[3,M]、……、[M,M]所构成的 KMP 树，则答案的变化量就是 f(1:M+1)、f(2:M+1)、……、f(3:M+1)，答案的变化量的变化量即为这些子串新添加一个叶子|M+1|的深度之和。结合以上性质，我们发现答案的变化量的变化量=字符串[1,M+1]中有多少子串作为[1,M+1]的后缀出现。

考虑使用后缀自动机维护，先离线构建出整个字符串的后缀自动机，对于将右端点扩展的每次操作首先更新每个节点的 end-set 集大小，需要查询的结果即为从后缀自动机上的 M+1 对应的节点向根一直走，每走到一个节点将答案加上节点的 end-set 集大小乘以节点包含的串数目。

因为数据随机，暴力的时间复杂度也是期望单次 O(logN)的。

时间复杂度：O(NlogN)。

期望得分：60 分（结合算法二）。

算法四：

使用动态树或者树链剖分+线段树维护信息代替暴力即可。

时间复杂度：O(NlogN)或 $O(Nlog^2N)$。

期望得分：100 分。